

Ly Gia Bao

Backend Java Engineer | Distributed Systems & Databases

lygiabaokg2002@gmail.com | [linkedin.com/in/baorlys](https://www.linkedin.com/in/baorlys) | baorlys.dev | github.com/baorlys

SUMMARY

Backend engineer with over two years on Java and Spring Boot services, since May 2024. I work across two products for the same fintech group: a licensed digital bank with 3 million customers, where I sit in the eVoucher and onboarding teams, and a consumer super app under development, where I build the satellite cluster of identity and session services. Recent work includes customer ID allocation running around 11,000 IDs a day with no duplicates, voucher issuance across partner APIs that stays correct under retry and redelivery, and a configurable step-up authentication engine. I also build the backend team's AI-assisted development tooling, which is part of why the client rated me top performer among the engineers my company has on site.

TECHNICAL SKILLS

Languages: Java 11–25, SQL, Python.

Java core: OOP, generics, collections internals, concurrency (executors, locks, memory visibility), virtual threads, JVM memory model, GC, classloading.

Frameworks: Spring Boot 3/4, Spring MVC, Spring Data JPA / Hibernate, Spring Security, Spring Batch, Spring Cloud Stream, OpenFeign, Resilience4j.

Databases: PostgreSQL (query tuning, indexing, partitioning, isolation levels, `FOR UPDATE SKIP LOCKED`), MySQL, MariaDB, Liquibase, Flyway, DynamoDB (single-table design, conditional writes), Elasticsearch.

Caching & messaging: Redis (distributed lock, cache-aside, L1 Caffeine, Pub/Sub), Apache Kafka (consumer groups, DLQ, redelivery), RabbitMQ.

Architecture: Microservices, DDD, Clean / Hexagonal, event-driven design, saga with compensating actions, transactional outbox, CQRS, idempotency, circuit breaker, retry with backoff.

Security: OAuth2 / OIDC, Keycloak (authenticator SPI, token exchange), JWT and JWKS, AWS Cognito, PII masking, KMS and Secrets Manager.

Platform & observability: Docker, Kubernetes (EKS), Helm, ArgoCD, GitHub Actions, Jenkins, Gradle, Maven, SonarQube, Micrometer, Prometheus, Grafana, OpenTelemetry.

AI-assisted development: Claude Code as daily tooling. Custom subagents, MCP servers (Jira, Confluence, SonarQube, LSP-backed code navigation), lifecycle hooks, reusable skills. Author of the backend team's internal Claude Code plugin.

Testing & process: JUnit 5, Mockito, Testcontainers, WireMock, JaCoCo, OpenAPI / Swagger. Scrum with Jira, code review, Confluence documentation.

EXPERIENCE

- **CMC Global** (client: a fintech group running a licensed digital bank and a consumer super app) Ho Chi Minh City
Oct 2025 – Present
Backend Engineer
 - **Digital bank, customer ID allocation: around 11,000 CIFs a day with no duplicates.** Replaced sequence-based issuance in the bank onboarding flow with per-customer-type pools, each carrying its own Feistel seed, prefix and ID length. The Feistel network yields non-predictable IDs in constant time with no pre-generation table and no uniqueness probe. Claims run through PostgreSQL `FOR UPDATE SKIP LOCKED` so concurrent pods skip locked rows instead of queueing behind them. Pools refill on a cron below a minimum threshold, with a `max-batches-per-run` cap as backpressure so a drained pool cannot stampede the database, a reclaim job for reservations orphaned by pod crashes, and Micrometer pool-depth gauges feeding alerts before a pool runs dry.
 - **Digital bank, eVoucher issuance.** Rebuilt voucher issuance as a provider-agnostic orchestration, one handler factory in front of two partner APIs and physical fulfilment, so adding a provider means adding a handler rather than a branch. Each issuance runs claim, issue, confirm, and a failure releases the claim as a compensating action instead of leaking inventory. Errors are split retryable and non-retryable, a scoped Feign retryer absorbs partner 5xx in process and Kafka redelivery is the outer retry, so the flow is safe to replay end to end. Per-customer and sliding-window quotas bound issuance.
 - **Super app, step-up authentication engine.** Own the challenge-chain service, which drives step-up authentication as versioned chains of challenges rather than hard-coded branches: assurance-level floors per customer tier, chain and action configuration stored as JSONB and promoted by Liquibase, and history that stays readable across versions. Put a Caffeine cache in front of the hot path after profiling showed a chain start costing three queries and a switch costing five, with TTL-bounded staleness across pods and local invalidation on config append.

- **Super app, OAuth2 and mini-app sessions.** Built the Keycloak-backed authentication service for the satellite cluster: account provisioning APIs, a custom Keycloak authenticator packaged as a Java SPI in a purpose-built image, and token exchange gated by an allowed-client policy. Designed the mini-app session flow where an init call establishes who the customer is and the exchange decides which mini-app APIs a host may reach, with a deterministic per-program pseudonym as the token subject instead of the internal customer id. Published JWKS with explicit `use` and `alg` so downstream verifiers select the right key.
- **Super app, satellite services.** Work across the rest of the cluster: MFA, device management, onboarding, configuration, partner integration, and the BFF for web and mobile. Started the onboarding service from scratch on Java 25 and Spring Boot 4 with a Clean Architecture split, modelling OTP behind a `ChallengeAuthenticator` port so swapping the challenge provider never reaches the domain, and delivered it whole: Liquibase schema, Kafka messaging, OpenAPI contract, WireMock and JUnit integration tests, and CI through DEV, SIT, UAT and STG.
- **Data modelling and SQL, both products.** Wrote 17 Liquibase changesets promoted from DEV through UAT to STG across the allocation, campaign and onboarding schemas, covering indexes, reference data and least-privilege grants per owning service. Cut write amplification on the hot allocation table by restructuring it, and dropped a dedup index that had become the throughput ceiling. On the NoSQL side, modelled the onboarding session as one DynamoDB aggregate under a single partition key holding KYC, ID card, NFC and face-check evidence, so one read serves the whole flow, with creation made idempotent by a conditional write. Integrated both products with the notification service fanning out to OTT, SMS and email.
- **AI-assisted development for the team.** Author of the backend team's internal Claude Code plugin. Its 14 skills turn our written conventions into checks that run by themselves: error handling and logging review, Liquibase changeset scaffolding, service scaffolding, and a pre-ship gate. Set up the tooling around it too, custom subagents, MCP servers for Jira, Confluence, SonarQube and LSP-backed code navigation, and lifecycle hooks, so generated code is reviewed against team standards rather than trusted. Review and delivery time dropped, convention defects stopped reaching reviewers, and minor bugs fell sharply. The process layer is portable to another company by swapping the convention files.
- **Platform, CI/CD and observability.** Onboarded services onto EKS with Helm charts reconciled by ArgoCD, using dedicated IRSA roles and KMS-backed secrets in place of in-chart credentials. Kafka topics, their DLQs and gateway exposure live in versioned config promoted per environment. Built GitHub Actions pipelines on Gradle and Jib, and instrumented the KYC and account-creation funnels with tagged Micrometer counters into Prometheus and Grafana, plus OTLP tracing.
- **Troubleshooting and team work.** Root-caused a thread-pinning defect in the AWS SDK v2 integration that was quietly capping request throughput in production. Led 3 engineers on the client-site squad for about six months, reviewing for concurrency, transaction boundaries and SOLID across a 60+ repository estate, and maintain an OpenAPI verifier Gradle plugin that fails CI when a contract breaks. Daily work with BA, QC and DevOps in Scrum.

- **ID Solutions (contract)**

Ho Chi Minh City

Software Engineer

Jun 2025 – Sep 2025

- Built a MariaDB to Elasticsearch sync on the transactional outbox pattern, which gave eventual consistency and strict ordering without dual-write risk.
- Rewrote the reporting queries and trimmed JVM allocation, taking Excel export from 20–30s down to 6–8s.
- Built the ledger module with the concurrency controls that keep balances correct across several funding sources, and a vendor-agnostic e-invoice signing layer for tax authority compliance.

- **NashTech Vietnam, bbv Vietnam**

Ho Chi Minh City

Software Engineer Intern (full-time, paid)

May 2024 – Jun 2025

- **NashTech** (Mar–Jun 2025): fixed N+1 queries in internal reporting with Hibernate DTO projections, 40% faster retrieval, and contributed to an asset management system and an online store.
- **bbv** (May–Dec 2024): worked on a multi-tenant digital asset management product with per-tenant isolation, moved heavy image and AI template processing onto RabbitMQ, and built a centralised notification service covering voice, SMS and email.

EDUCATION

- **Ton Duc Thang University**

Ho Chi Minh City

Bachelor of Software Engineering

Oct 2020 – 2026

- All coursework and the thesis are complete. Awaiting degree conferral in 2026.

ADDITIONAL

- **ICPC programming team** (2022–2023): regional competitive programming.
- **Languages:** Vietnamese native, English at working level. Daily work on English specifications and vendor API documentation.